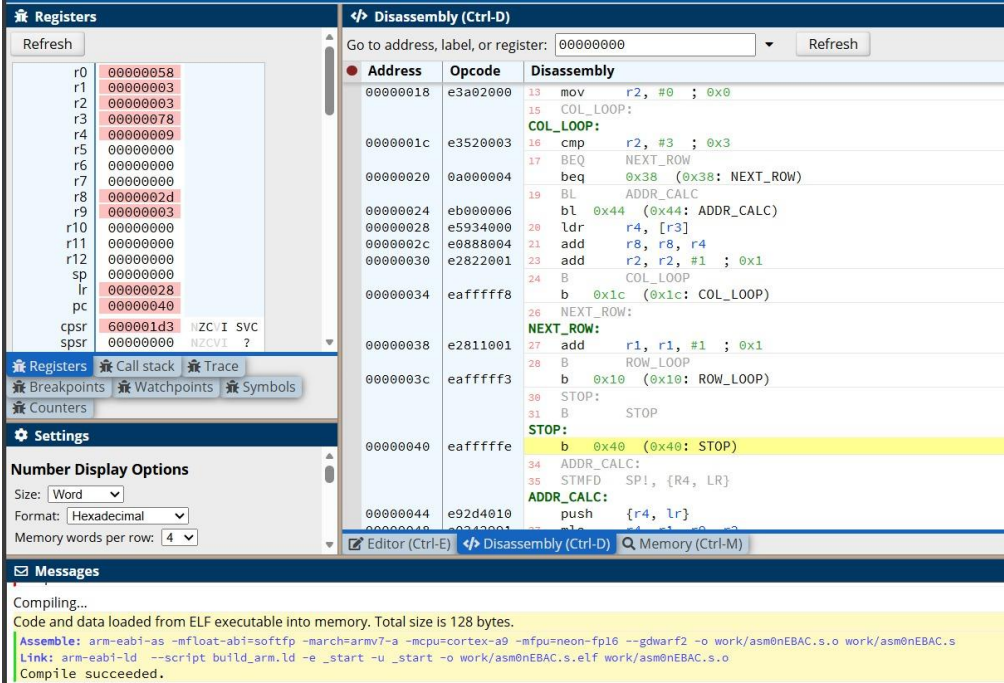


Department of Computer Science & Engineering

Microprocessor & Computer Architecture

UE24CS251B

MPCA Lab Programs - 5

Name: Dhriti Kamani	SRN: PES1UG24AM085	Section:B
1	Write an ARM assembly language program to compute the sum of all elements in a 3×3 matrix stored in memory. The matrix elements are stored in row-major order as a one-dimensional array. The program should: • Use nested loops to iterate over rows and columns. • Compute the effective memory address of each matrix element using a separate subroutine. • Load each matrix element from memory and accumulate the total sum in a register. • Store or maintain the final sum in a designated register before program termination. The program must use: • A subroutine to calculate the address of a[i][j] • Stack operations (STMFD, LDMFD) to preserve register values • Multiply-Accumulate (MLA) for address computation Program and Results Screenshot:  The screenshot displays the ARM development environment. On the left, the 'Registers' panel shows the state of various registers, with R0 containing 00000058. The 'Disassembly (Ctrl-D)' panel shows the assembly code for the program, including nested loops for rows and columns, address calculation, and accumulation. The 'Messages' panel at the bottom shows the compilation process, indicating that the code and data were loaded into memory and the compilation succeeded. <pre> .global _start _start: LDR R0, =MAT MOV R1, #0 MOV R8, #0 </pre>	

	<pre> MOV R9, #3 ROW_LOOP: CMP R1, #3 BEQ STOP MOV R2, #0 COL_LOOP: CMP R2, #3 BEQ NEXT_ROW BL ADDR_CALC LDR R4, [R3] ADD R8, R8, R4 ADD R2, R2, #1 B COL_LOOP NEXT_ROW: ADD R1, R1, #1 B ROW_LOOP STOP: B STOP ADDR_CALC: STMFD SP!, {R4, LR} MLA R4, R1, R9, R2 LSL R4, R4, #2 ADD R3, R0, R4 LDMFD SP!, {R4, PC} MAT: .word 1, 2, 3 .word 4, 5, 6 .word 7, 8, 9 </pre>
2	Write an ARM assembly program to transfer a 3×3 matrix stored in memory location A to memory location C. Use a subroutine to compute element addresses and stack operations to save and restore registers. The source matrix is stored as a one-dimensional array in row-major order. The program should: • Use nested loops to traverse rows and columns of the matrix. • Compute the effective address of each matrix element using a separate subroutine. • Load each element from the source matrix and store it into the destination matrix. • Use stack operations to preserve register values during subroutine calls. • Ensure that all matrix elements are copied correctly before program termination. Program and Results Screenshot:

Registers

Refresh

r0	00000058
r1	00000003
r2	00000003
r3	00000078
r4	00000009
r5	00000000
r6	00000000
r7	00000000
r8	0000002d
r9	00000003
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000028
pc	00000040
cpsr	600001d3 NZCVI SVC
spsr	00000000 NZCVI ?

[Registers](#)
[Call stack](#)
[Trace](#)

[Breakpoints](#)
[Watchpoints](#)
[Symbols](#)

[Counters](#)

Settings

Number Display Options

Size: Word

Format: Hexadecimal

Memory words per row: 4

Disassembly (Ctrl-D)

Go to address, label, or register: 00000000 Refresh

Address	Opcode	Disassembly
00000018	e3a02000	13 mov r2, #0 ; 0x0
		15 COL_LOOP:
0000001c	e3520003	16 cmp r2, #3 ; 0x3
		17 BEQ NEXT_ROW
00000020	0a000004	21 beq 0x38 (0x38: NEXT_ROW)
		19 BL ADDR_CALC
00000024	eb000006	b1 0x44 (0x44: ADDR_CALC)
00000028	e5934000	20 ldr r4, [r3]
0000002c	e0888004	21 add r8, r8, r4
00000030	e2822001	23 add r2, r2, #1 ; 0x1
		24 B COL_LOOP
00000034	ea000000	b 0x1c (0x1c: COL_LOOP)
		26 NEXT_ROW:
00000038	e2811001	27 add r1, r1, #1 ; 0x1
		28 B ROW_LOOP
0000003c	ea000000	b 0x10 (0x10: ROW_LOOP)
		30 STOP:
		31 B STOP
00000040	ea000000	b 0x40 (0x40: STOP)
		34 ADDR_CALC:
		35 STMFD SP!, {R4, LR}
00000044	e92d4010	ADDR_CALC: push {r4, lr}
00000048	e92d4010	push {r4, lr}

[Editor \(Ctrl-E\)](#)
[Disassembly \(Ctrl-D\)](#)
[Memory \(Ctrl-M\)](#)

Messages

CPDiator has started with system **AKMIV7** generic containing a **AKMIV7** processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 128 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfp=neon-fp16 --gdwarf2 -o work/asmmdm9qu.s.o work/asmmdm9qu.s
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmmdm9qu.s.elf work/asmmdm9qu.s.o
Compile succeeded.

```

.global _start
_start:
    LDR    R0, =MAT
    MOV    R1, #0
    MOV    R8, #0
    MOV    R9, #3
ROW_LOOP:
    CMP    R1, #3
    BEQ    STOP
    MOV    R2, #0
COL_LOOP:
    CMP    R2, #3
    BEQ    NEXT_ROW
    BL     ADDR_CALC
    LDR    R4, [R3]
    ADD    R8, R8, R4
    ADD    R2, R2, #1
    B      COL_LOOP
NEXT_ROW:
    ADD    R1, R1, #1
    B      ROW_LOOP
STOP:
    B      STOP
ADDR_CALC:
    STMFD  SP!, {R4, LR}
    MLA    R4, R1, R9, R2
    LSL    R4, R4, #2
    ADD    R3, R0, R4

```

LDMFD SP!, {R4, PC}

MAT:

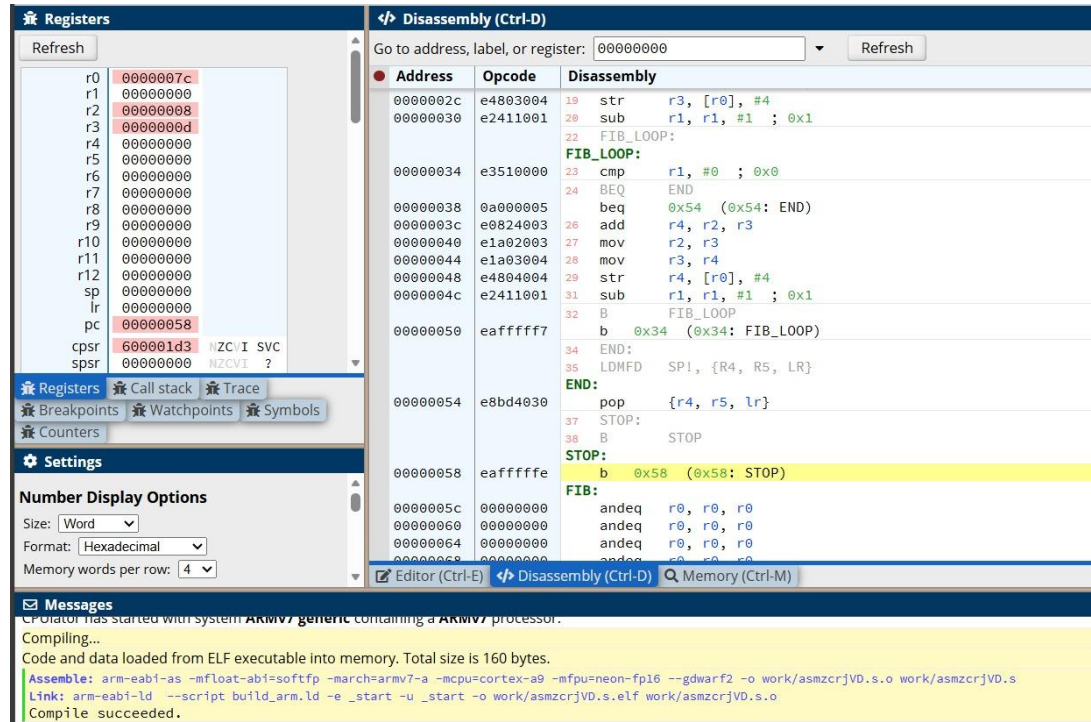
.word 1, 2, 3

.word 4, 5, 6

.word 7, 8, 9

- 3 **Generate the first N Fibonacci numbers and store them in memory while using stack operations for register preservation.**

Program and Results Screenshot:



The screenshot shows a debugger interface with the following components:

- Registers:** A list of registers (r0-r15, sp, lr, pc, cpsr, spsr) with their current values. The PC register is highlighted at 00000058.
- Disassembly (Ctrl-D):** A table showing the disassembled code. The code includes instructions for setting up the stack, calculating Fibonacci numbers, and storing them in memory. The disassembly is as follows:

Address	Opcode	Disassembly
0000002c	e4803004	19 str r3, [r0], #4
00000030	e2411001	20 sub r1, r1, #1 ; 0x1
00000034	e3510000	22 FIB_LOOP: cmp r1, #0 ; 0x0
00000038	0a000005	23 BEQ END (0x54: END)
0000003c	e0824003	24 add r4, r2, r3
00000040	e1a02003	26 mov r2, r3
00000044	e1a03004	27 mov r3, r4
00000048	e4804004	28 str r4, [r0], #4
0000004c	e2411001	29 sub r1, r1, #1 ; 0x1
00000050	ea000000	30 B FIB_LOOP (0x34: FIB_LOOP)
00000054	e8bd4030	34 END: pop {r4, r5, lr}
00000058	ea000000	35 LDMFD SP!, {R4, R5, LR}
0000005c	00000000	37 STOP: STOP
00000060	00000000	38 B STOP
00000064	00000000	39 FIB: andeq r0, r0, r0
00000068	00000000	andeq r0, r0, r0
0000006c	00000000	andeq r0, r0, r0
00000070	00000000	andeq r0, r0, r0
- Messages:** A log of compiler messages showing the compilation of the program. The messages include:


```

      Compiler has started with system ARMv7-generic containing a ARMv7 processor.
      Compiling...
      Code and data loaded from ELF executable into memory. Total size is 160 bytes.
      Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mcpu=neon-fp16 -gdwarf2 -o work/asmzcrjVD.s.o work/asmzcrjVD.s
      Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmzcrjVD.s.elf work/asmzcrjVD.s.o
      Compile succeeded.
      
```

```

.global _start
_start:
    LDR    R0, =FIB
    MOV    R1, #8
    MOV    R2, #0
    MOV    R3, #1
    STMFD  SP!, {R4, R5, LR}
    CMP    R1, #0
    BEQ    END
    STR    R2, [R0], #4
    SUB    R1, R1, #1
    CMP    R1, #0
    BEQ    END
    STR    R3, [R0], #4
    SUB    R1, R1, #1
FIB_LOOP:
    CMP    R1, #0
    BEQ    END
    ADD    R4, R2, R3
    MOV    R2, R3
    MOV    R3, R4
    STR    R4, [R0], #4
  
```

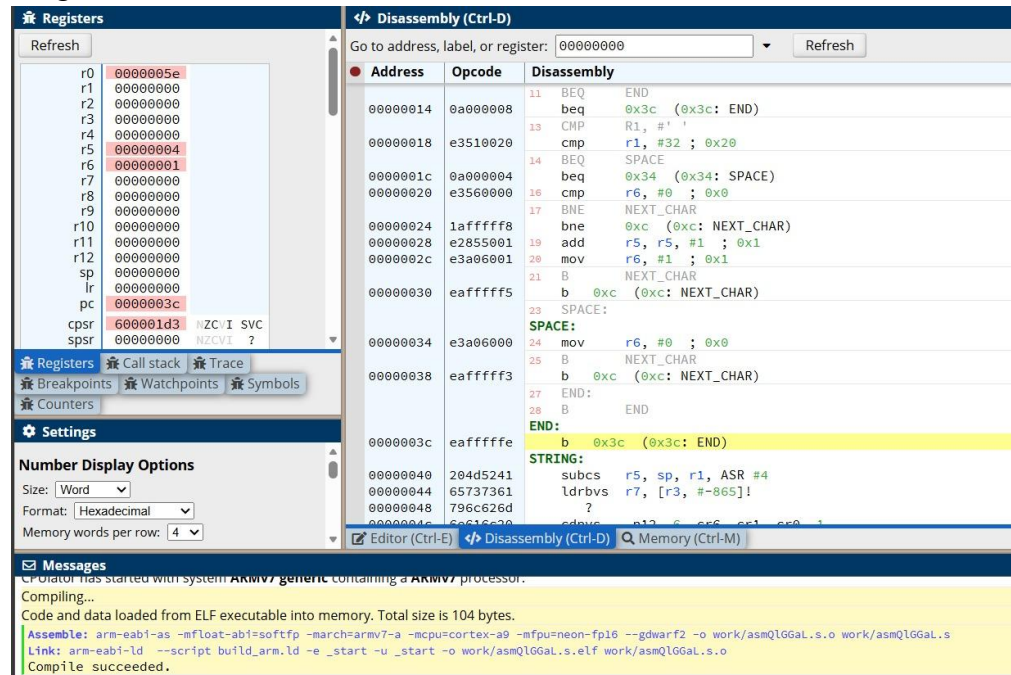
```

SUB    R1, R1, #1
B      FIB_LOOP
END:
LDMFD  SP!, {R4, R5, LR}
STOP:
B      STOP
FIB:
.space 64

```

4 **Write an ARM ALP to count the number of words in a string.**

Program and Results Screenshot:



The screenshot shows the ARM Disassembler interface. The left pane displays the registers, and the right pane shows the disassembled code. The code is as follows:

```

00000014 0a000008 beq     0x3c (0x3c: END)
00000018 e3510020 cmp     R1, #32 ; 0x20
0000001c 0a000004 beq     0x34 (0x34: SPACE)
00000020 e3560000 cmp     R6, #0 ; 0x0
00000024 1afffff8 bne     0xc (0xc: NEXT_CHAR)
00000028 e2855001 add     R5, R5, #1 ; 0x1
0000002c e3a06001 mov     R6, #1 ; 0x1
00000030 eaffffff b       0xc (0xc: NEXT_CHAR)
00000034 e3a06000 mov     R6, #0 ; 0x0
00000038 eaffffff b       0xc (0xc: NEXT_CHAR)
0000003c eaffffff b       0x3c (0x3c: END)

```

The bottom pane shows the compilation and linking process, indicating that the program was successfully compiled and linked.

```

.global _start
_start:
    LDR    R0, =STRING
    MOV    R5, #0
    MOV    R6, #0
NEXT_CHAR:
    LDRB   R1, [R0], #1
    CMP    R1, #0
    BEQ    END
    CMP    R1, #' '
    BEQ    SPACE
    CMP    R6, #0
    BNE    NEXT_CHAR
    ADD    R5, R5, #1
    MOV    R6, #1
    B      NEXT_CHAR
SPACE:
    MOV    R6, #0
    B      NEXT_CHAR
END:
    B      END

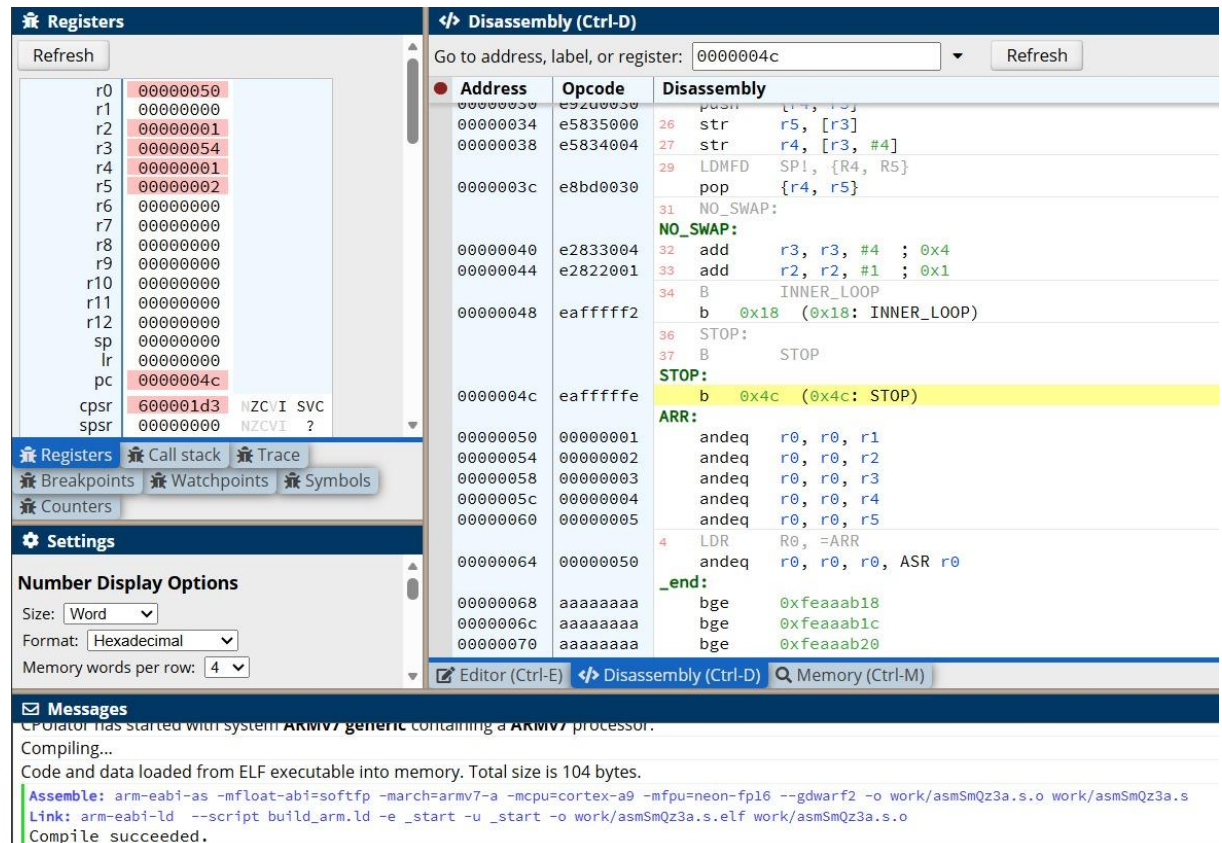
```


STRING:
.asciz "ARM assembly language program"

5 ASSIGNMENT QUESTION 1

Write an ALP using conditional ARM instructions to sort an array of numbers using Bubble Sort Algorithm. Use STMFD to save registers before swap and LDMFD to restore the registers

Program and Results Screenshot:



Registers

Register	Value
r0	00000050
r1	00000000
r2	00000001
r3	00000054
r4	00000001
r5	00000002
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	0000004c
cpsr	600001d3
spsr	00000000

Disassembly (Ctrl-D)

Address	Opcode	Disassembly
00000030	e9200030	push {r4, r5}
00000034	e5835000	str r5, [r3]
00000038	e5834004	str r4, [r3, #4]
0000003c	e8bd0030	LDMFD SP!, {R4, R5}
00000040	e2833004	pop {r4, r5}
00000044	e2822001	NO_SWAP:
00000048	ea000000	add r3, r3, #4 ; 0x4
0000004c	ea000000	add r2, r2, #1 ; 0x1
00000050	ea000000	B INNER_LOOP
00000054	ea000000	b 0x18 (0x18: INNER_LOOP)
00000058	ea000000	STOP:
0000005c	ea000000	B STOP
00000060	ea000000	STOP:
00000064	ea000000	b 0x4c (0x4c: STOP)
00000068	ea000000	ARR:
0000006c	ea000000	andeq r0, r0, r1
00000070	ea000000	andeq r0, r0, r2
00000074	ea000000	andeq r0, r0, r3
00000078	ea000000	andeq r0, r0, r4
0000007c	ea000000	andeq r0, r0, r5
00000080	ea000000	LDR R0, =ARR
00000084	ea000000	andeq r0, r0, r0, ASR r0
00000088	ea000000	_end:
0000008c	ea000000	bge 0xfeaab18
00000090	ea000000	bge 0xfeaab1c
00000094	ea000000	bge 0xfeaab20

Messages

CP Editor has started with system ARMv7 generic containing a ARMv7 processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 104 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asmSmQz3a.s.o work/asmSmQz3a.s

Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmSmQz3a.s.elf work/asmSmQz3a.s.o

Compile succeeded.

.global _start

_start:

LDR R0, =ARR

MOV R1, #5

OUTER_LOOP:

SUBS R1, R1, #1

BEQ STOP

MOV R2, #0

LDR R3, =ARR

INNER_LOOP:

CMP R2, R1

BEQ OUTER_LOOP

LDR R4, [R3]

LDR R5, [R3, #4]

CMP R4, R5

BLE NO_SWAP

```

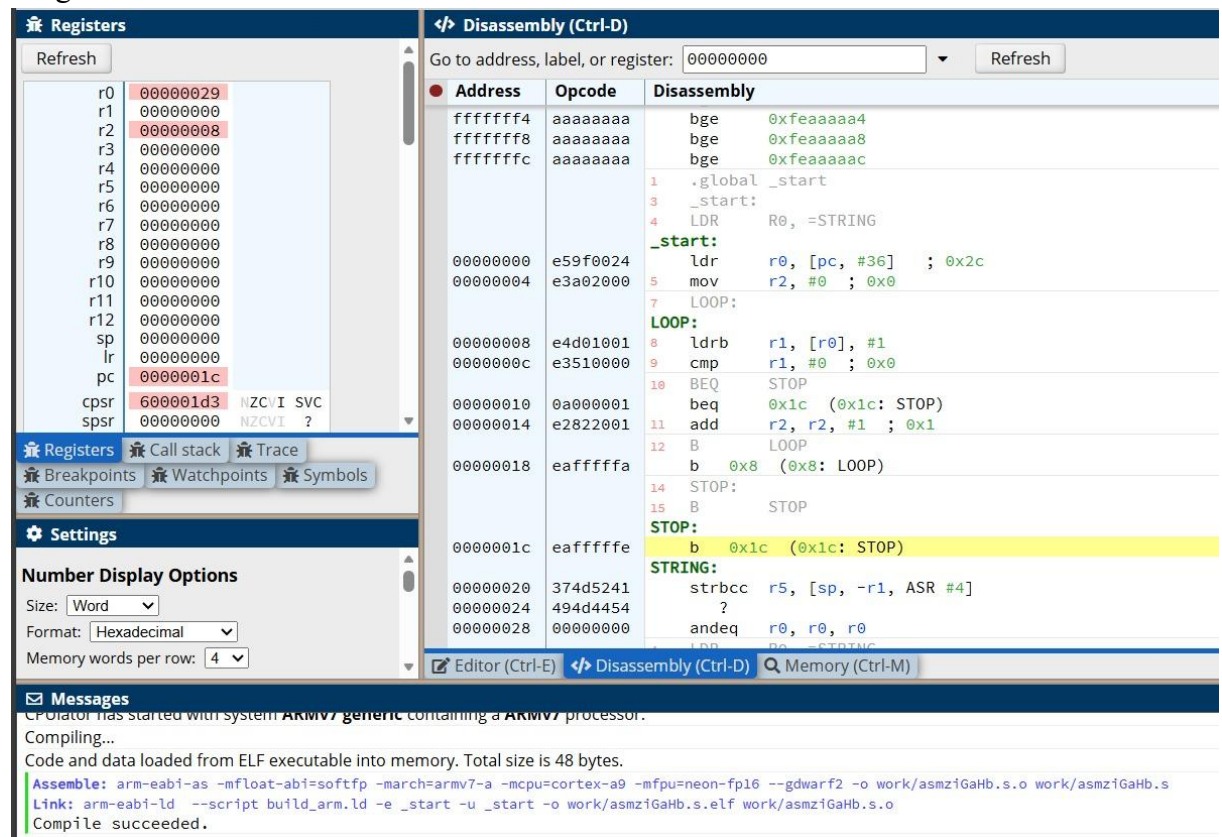
STMFD SP!, {R4, R5}
STR    R5, [R3]
STR    R4, [R3, #4]
LDMFD SP!, {R4, R5}
NO_SWAP:
    ADD    R3, R3, #4
    ADD    R2, R2, #1
    B      INNER_LOOP
STOP:
    B      STOP
ARR:
    .word  5, 1, 4, 2, 3

```

6 ASSIGNMENT QUESTION 2

Write an ALP using ARM7TDMI to find the length of string

Program and Results Screenshot:



The screenshot shows the ARM7TDMI emulator interface. The left pane displays the registers, with the PC register at address 0000001c. The right pane shows the disassembly of the program, starting at address 00000000. The program code is as follows:

```

.global _start
_start:
    LDR    R0, =STRING
    MOV    R2, #0
LOOP:
    LDRB   R1, [R0], #1
    CMP    R1, #0
    BEQ    STOP
    BEQ    0x1c (0x1c: STOP)
    ADD    R2, R2, #1
    B      LOOP
STOP:
    B      STOP
STOP:
    B      0x1c (0x1c: STOP)
STRING:
    strbcc r5, [sp, -r1, ASR #4]
    ?
    andeq  r0, r0, r0

```

The Messages pane at the bottom shows the following output:

```

C:\Orion\ has started with system ARMv7 generic containing a ARMv7 processor.
Compiling...
Code and data loaded from ELF executable into memory. Total size is 48 bytes.
Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mfpu=neon-fp16 --gdwarf2 -o work/asmziGaHb.s.o work/asmziGaHb.s
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asmziGaHb.s.elf work/asmziGaHb.s.o
Compile succeeded.

```

.global _start

_start:

```

LDR    R0, =STRING
MOV    R2, #0

```

LOOP:

```

LDRB   R1, [R0], #1
CMP    R1, #0
BEQ    STOP

```

	<pre>ADD R2, R2, #1 B LOOP STOP: B STOP STRING: .asciz "ARM7TDMI"</pre>
--	--